Discovery Executive Summary

Project: test-disocvery · **Generated:** 18/08/2026, 19:05:06

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 7 discovery analyses run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

Portfolio Overview

#	Analysis	Overall Rating
1	Architecture & Design Analysis	—
2	Code Quality & Complexity Analysis	—
3	Frontend Modernization Analysis	—
4	Backend Modernization Analysis	—
5	Testing & Quality Assurance Analysis	—
6	Security Analysis	—
7	Performance & Sustainability Analysis	—

---

1. Architecture & Design Analysis

EXECUTIVE SUMMARY

The Klearcom platform is a monorepo with a Laravel 12 backend (6 API controllers, 4 Eloquent models, 2 services), a React 19 SPA frontend (4 page components, 4 shared components, 1 custom hook, 1 Zustand store), and a Node/Express dev-API (6 source files). Backend controllers are lean on average (74 LOC) but 4 of 6 access Eloquent models directly — there is no repository layer, and business logic (reachability calculations, tree building) is duplicated across controllers and services in 3 independent locations. The `LegacyReportController` crosses both Discovery and Connect domain boundaries, reading all four models, while `DashboardController` does the same for aggregation. The `LegacyDataMapper` uses PHP `extract()` — a dynamic-variable anti-pattern that creates implicit coupling. On the frontend, all 4 page components and 1 legacy class component make direct `api.*` calls with no service/data-access abstraction layer; the `LegacyMonitorPoller` is a class component with an interval memory leak. The two most urgent risks are (1) the missing repository pattern which scatters ORM access across 32 call sites, and (2) the cross-domain coupling in `LegacyReportController` and `DashboardController` that prevents independent evolution of Discovery and Connect modules. Layers covered: backend (26 PHP files), frontend (15 TS/TSX/JSX files), dev-api (6 JS files).

1.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	Good	Moderate	High Risk	Measured	Rating
H1	Fat Controllers	Avg LOC per controller	<150	150–300	>300	74 LOC avg (max 102)	Good
H2	Missing Service Layer	Controllers accessing repos/models	<15	15–30	>30	25 direct model calls in controllers	Moderate
H3	Missing Repository Pattern	Direct DB access points outside repos	<15	15–30	>30	32 (25 controllers + 7 services)	High Risk
H4	Circular Dependencies	Dependency cycles	0	1–3	>3	0	Good
H5	Shared Utility Abuse	Utility files w/ business logic	0	1–5	>5	1 (LegacyDataMapper)	Moderate
H6	Direct SQL in Controllers	ORM compliance %	>90%	60–90%	<60%	100% (all Eloquent, no raw SQL)	Good
H7	God Classes	Largest class/file LOC	<500	500–600	>600	276 LOC (dev-api server.js); 222 LOC (ConnectPage.tsx); 165 LOC (MongoService.php)	Good
H8	Domain Boundary Violations	Cross-domain access points	0	1–5	>5	4 (LegacyReportController: 4 cross-domain models; DashboardController:	Moderate

						2)	
H9	Shared Database Coupling	Tables shared across domains	<30%	30–40%	>40%	~33% — 2 of 4 models queried by cross-domain controllers	Moderate
H10	Duplicated Business Logic (additional)	Independent copies of same algorithm	0	1–2	>2	3 copies of reachability formula + 3 copies of buildTree	Moderate
H11	Missing Frontend Service Layer (additional)	Page components with direct API calls	0	1–3	>3	5 components with 14 total direct api.* calls	Moderate

1.4 Actions Required

Hotspot	Action	Rating	Priority
H3. Missing Repository Pattern	Introduce `ConnectMonitorRepository` and `DiscoveryJobRepository` interfaces with Eloquent implementations; bind in `AppServiceProvider`; replace all 32 direct `Model::` calls	High Risk	High
H2. Missing Service Layer	Create `ConnectReachabilityService`, `DiscoveryTreeService`, `DashboardKpiService`; move business logic out of 4 controllers	Moderate	High
H10. Duplicated Business Logic	Consolidate 3 copies of reachability formula and 3 copies of `buildTree()` into single canonical service methods	Moderate	High
H8. Domain Boundary Violations	Split `LegacyReportController` by domain; route cross-domain reads through service interfaces	Moderate	Medium
H9. Shared Database Coupling	Define table ownership per domain; expose read-only service APIs for cross-domain consumers	Moderate	Medium
H5. Shared Utility Abuse	Remove `extract()` from `LegacyDataMapper` and `LegacyReportController`; move mapping logic to domain service	Moderate	Medium
H11. Missing Frontend Service Layer	Create `connectService.ts` / `discoveryService.ts`; extract domain hooks; convert `LegacyMonitorPoller` to functional component with cleanup	Moderate	Medium

1.5 Expected Outcomes

- **Testability:** Repository interfaces enable unit testing of services without a database; frontend service layer enables mocking API calls in component tests.
- **Single source of truth:** Consolidating the reachability formula and `buildTree()` into canonical service methods eliminates the risk of divergent business logic across endpoints.
- **Independent module evolution:** Enforcing domain boundaries (Discovery vs. Connect) through service interfaces means schema changes in one domain cannot silently break the other.
- **Onboarding velocity:** New contributors can work on Discovery or Connect independently without needing to understand both domains — bounded contexts reduce the knowledge surface area.
- **Migration readiness:** Repository abstraction + domain services position the codebase for future extraction into separate deployable services, should the platform scale beyond a monolith.","stop_reason":"end_turn","session_id":"a718565e-0393-476a-8617-856b70cd46ec","total_cost_usd":2.3541065,"usage":{"input_tokens":18,"cache_creation_input_tokens":98918,"cache_read_input_tokens":1147903,"output_tokens":31228,"server_tool_use":{"web_search_requests":0,"web_fetch_requests":0},"service_tier":"standard","cache_creation":{"ephemeral_1h_input_tokens":98918,"ephemeral_5m_input_tokens":0},"inference_geo":"not_available","iterations":[{"input_tokens":1,"output_tokens":1999,"cache_read_input_tokens":106182,"cache_creation_input_tokens":10409,"cache_creation":{"ephemeral_5m_input_tokens":0,"ephemeral_1h_input_tokens":10409},"type":"message"}],"speed":"standard"},"modelUsage":{"claude-haiku-4-5-20251001":{"inputTokens":10105,"outputTokens":16,"cacheReadInputTokens":0,"cacheCreationInputTokens":0,"webSearchRequests":0,"costUSD":0.010185,"contextWindow":200000,"maxOutputTokens":3200
  opus-4-6":{"inputTokens":18,"outputTokens":31228,"cacheReadInputTokens":1147903,"cacheCreationInputTokens":98918,"webSearchRequests":0,"costUSD":2.3439215,"contextWindow":200000,"maxOutput
  [],"terminal_reason":"completed","fast_mode_state":"off","uuid":"f330f0c5-bea8-4977-88b2-fd4b9b4d5a8e"}

---

2. Code Quality & Complexity Analysis

EXECUTIVE SUMMARY

The Klearcom platform codebase is compact (~~3 000 LOC across 50 files) and largely well-structured, with no function or method exceeding a cyclomatic complexity of 20. However, one frontend component — `ConnectPage.tsx` — exceeds the 200-LOC single-function threshold at ~210 source lines, concentrating form state, three TanStack Query hooks, event handlers, and two data tables in a single render function. Business-logic duplication sits at the Moderate boundary (~~5 %): the `buildTree` algorithm is copy-pasted across `DiscoveryController`, `LegacyReportController`, and the dev-API `store.js`, while the reachability-rate calculation is repeated in `RealTimeTestService`, `ConnectController`, and the dev-API `realtime.js` — the latter already flagged inline as a duplicate. Refactoring these two areas will yield the highest return on code quality.

2.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	Good	Moderate	High	Measured	Rating

					Risk		
H1	High Cyclomatic Complexity	Methods with complexity >20	<30	30–40	>40	0 methods >20 (highest ~15 in ConnectPage.tsx; backend methods all <10)	Good
H2	Large Functions	Largest function LOC	<100	100–200	>200	~210 LOC (ConnectPage.tsx); DiscoveryPage.tsx ~166 LOC; all backend methods <60 LOC	High Risk
H3	Business Logic Duplication	Duplicated business logic %	<5%	5–10%	>10%	~5% (buildTree ×3, reachability calc ×3, document serialize ×2)	Moderate

No additional hotspots beyond the standard set were observed.

Hotspot Score breakdown

Component	Weight	Sub-score (0–100)	Weighted
Cyclomatic Complexity	50%	20	10.0
Function Size	30%	70	21.0
Business Logic Duplication	20%	38	7.6
Hotspot Score	**100%**		**39 / 100**

2.4 Actions Required

Hotspot	Action	Rating	Priority
H2 — Large Functions	Decompose `ConnectPage.tsx` (~~210 LOC) into sub-components (~~ ~~`MonitorForm`~~ , ~~`MonitorTable`~~ , ~~`CheckHistoryPanel`~~ , ~~`TranscriptsPanel`~~ ~~) and custom hooks; apply same decomposition to~~ ~~`DiscoveryPage.tsx`~~ ~~(~~166 LOC)	High Risk	Critical
H3 — Business Logic Duplication	Consolidate `buildTree` into a `TreeService` or model static method; extract reachability calculation into `ReachabilityService` or `ConnectMonitor::computeReachability()` ; remove inline copies from `ConnectController` and `LegacyReportController`	Moderate	High

2.5 Expected Outcomes

- **Lower defect risk on monitor status:** A single reachability-calculation service eliminates the risk of divergent alert thresholds across three code paths (RealTimeTestService, ConnectController, dev-API realtime.js).
- **Faster code reviews:** Splitting ConnectPage (~~210 LOC) and DiscoveryPage (~~166 LOC) into focused sub-components (each <80 LOC) reduces review scope and merge-conflict surface.
- **Safer refactors:** Removing the duplicated `buildTree` copies means IVR tree changes propagate automatically to both the discovery and legacy-report endpoints.
- **Easier onboarding:** New developers can understand each sub-component in isolation instead of tracing a 210-line function with three query hooks, two tables, and conditional rendering.
- **Improved testability:** Extracted hooks (`useConnectQueries` , `useMonitorForm`) and services (`ReachabilityService` , `TreeService`) can be unit-tested independently without rendering full page components.","stop_reason":"end_turn","session_id":"aa15eb01-e01f-4705-b891-5388ccb38547","total_cost_usd":2.1763269999999997,"usage":{"input_tokens":17,"cache_creation_input_tokens":105085,"cache_read_input_tokens":915782,"output_tokens":26367,"server_tool_use":{"web_search_requests":0,"web_fetch_requests":0},"service_tier":"standard","cache_creation":{"ephemeral_1h_input_tokens":105085,"ephemeral_5m_input_tokens":0},"inference_geo":"not_available","iterations":[{"input_tokens":1,"output_tokens":1488,"cache_read_input_tokens":99265,"cache_creation_input_tokens":5820,"cache_creation":{"ephemeral_5m_input_tokens":0,"ephemeral_1h_input_tokens":5820},"type":"message"}],"speed":"standard"},"modelUsage":{"claude-haiku-4-5-20251001":{"inputTokens":8241,"outputTokens":17,"cacheReadInputTokens":0,"cacheCreationInputTokens":0,"webSearchRequests":0,"costUSD":0.008326,"contextWindow":200000,"maxOutputTokens":32000}
opus-4-6":{"inputTokens":17,"outputTokens":26367,"cacheReadInputTokens":915782,"cacheCreationInputTokens":105085,"webSearchRequests":0,"costUSD":2.1680009999999994,"contextWindow":200000,"
[],"terminal_reason":"completed","fast_mode_state":"off","uuid":"de2d42c6-61ab-49df-8751-a02926e32c63"}

---

3. Frontend Modernization Analysis

EXECUTIVE SUMMARY

The Klearcom frontend is a compact React 19 + TypeScript single-page application consisting of 13 source files and 11 component definitions, built with a modern Vite toolchain and well-chosen library stack (Zustand for client state, React Query for server cache). The most severe gap is **significant structural and code duplication** between the two primary feature pages — `ConnectPage` and `DiscoveryPage` — which share near-identical form layouts, query patterns, invalidation logic, and MongoDB transcript rendering blocks that should be extracted into shared components and hooks. A legacy class-based component (`LegacyMonitorPoller.jsx`) with an **uncleared interval leak** and a `LegacyDashboardWidget` that **throws unhandled errors without an Error Boundary** represent resource safety and crash-resilience risks. Component sizes are well within healthy limits and state management is clean and scoped.

3.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	Good	Moderate	High Risk	Measured	Rating
H1	UI Component Duplication	Duplicate components %	<5%	5–10%	>10%	~33% (3 of 9 component files)	High Risk
H2	Legacy Class-Based Components	Modern component adoption %	>90%	70–90%	<70%	91% (10 of 11 definitions)	Good
H3	Massive Components	Largest component LOC	<400	400–500	>500	222 LOC (ConnectPage.tsx)	Good
H4	Global State Dependencies	Components reading global state %	<30%	30–60%	>60%	18% (2 of 11 components)	Good
H5	Complex State Management	Max prop-drilling depth	<3	3–5	>5	1–2 levels	Good
H6	Missing Error Boundaries (additional)	Error boundary coverage (%, target 100% of throw-capable subtrees)	>80%	50–80%	<50%	0% (no ErrorBoundary in codebase)	High Risk
H7	Resource / Memory Leak (additional)	Components with uncleared timers/subscriptions (count, target 0)	0	1	>1	1 (LegacyMonitorPoller)	Moderate

3.4 Actions Required

Hotspot	Action	Rating	Priority
H1 — UI Component Duplication	Extract shared `TranscriptList` component, create `useModulePage` hook to deduplicate page structure, delete `LegacyDashboardWidget.tsx`	High Risk	Critical
H6 — Missing Error Boundaries	Add `ErrorBoundary` at app root and per-route level; replace `throw new Error` with rendered error state	High Risk	High
H7 — Resource / Memory Leak	Convert `LegacyMonitorPoller` to functional component with `useQuery` refetchInterval, or add `componentWillUnmount` cleanup	Moderate	Medium

3.5 Expected Outcomes

- **Shared component library reduces duplication:** Extracting `TranscriptList` and `ModulePageLayout` eliminates ~33% structural duplication and makes adding new feature modules a configuration exercise rather than a copy-paste operation.
- **Error Boundaries improve resilience:** Per-route boundaries ensure a failing API in one module cannot crash the entire platform — critical for a voice monitoring dashboard that must stay available.
- **Functional component conversion improves safety:** Replacing the class-based `LegacyMonitorPoller` with a `useQuery` -based hook eliminates the interval memory leak and aligns with the codebase's established React Query patterns.
- **Deletion of legacy widget reduces maintenance surface:** Removing `LegacyDashboardWidget` eliminates redundant network calls and a throw-without-boundary crash vector.
- **Consistent patterns accelerate onboarding:** A single page layout pattern with hooks-based data fetching gives new contributors one clear way to build feature pages.","stop_reason":"end_turn","session_id":"4e19afb8-a3d4-48ce-80a3-b70d97cda1d5","total_cost_usd":1.7082904999999997,"usage":{"input_tokens":16,"cache_creation_input_tokens":85918,"cache_read_input_tokens":699709,"output_tokens":19670,"server_tool_use":{"web_search_requests":0,"web_fetch_requests":0},"service_tier":"standard","cache_creation":{"ephemeral_1h_input_tokens":85918,"ephemeral_5m_input_tokens":0},"inference_geo":"not_available","iterations":[{"input_tokens":1,"output_tokens":1352,"cache_read_input_tokens":80268,"cache_creation_input_tokens":5650,"cache_creation":{"ephemeral_5m_input_tokens":0,"ephemeral_1h_input_tokens":5650},"type":"message"}],"speed":"standard"},"modelUsage":{"claude-haiku-4-5-20251001":{"inputTokens":7346,"outputTokens":16,"cacheReadInputTokens":0,"cacheCreationInputTokens":0,"webSearchRequests":0,"costUSD":0.007426,"contextWindow":200000,"maxOutputTokens":32000}
opus-4-6":{"inputTokens":16,"outputTokens":19670,"cacheReadInputTokens":699709,"cacheCreationInputTokens":85918,"webSearchRequests":0,"costUSD":1.7008644999999996,"contextWindow":200000,"n
[],"terminal_reason":"completed","fast_mode_state":"off","uuid":"6d536e0c-debc-4f30-ae31-da7543f840bc"}

---

4. Backend Modernization Analysis

EXECUTIVE SUMMARY

The Klearcom platform is a monorepo with two parallel backend stacks: a PHP 8.3 / Laravel 12 application serving the production API (6 controllers, 19 endpoints, MariaDB + MongoDB) and a Node.js / Express 4.21 dev-api mirroring every endpoint with an in-memory store for local development (18 endpoints). Three occurrences of PHP `extract()` were found — two in `LegacyDataMapper` and one in `LegacyReportController` — where raw request data or untrusted arrays are unpacked into local variables without validation. The codebase has no OpenAPI specification, no API versioning, no contract tests, and no API linting despite exposing a significant REST surface. The dev-api duplicates every Laravel endpoint without a

shared contract, which means the two backends can diverge silently — and already have (a `bulk-import` endpoint exists only in dev-api, and the `checks` response shape differs). Overall, the backend carries moderate risk driven by the absence of API governance across both stacks.

4.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	Good	Moderate	High Risk	Measured	Rating
H1	Dynamic Variable Creation	Dynamic-var-from-input occurrences	0	1–10	>10	3	Moderate
H2	API Sprawl	Documented & governed endpoints %	>90%	80–90%	<80%	~49% (18 of 37 endpoints duplicated across two backends with no shared contract; 1 dev-only endpoint with no production equivalent)	High Risk
H3	Missing API Governance	Governance compliance %	100%	90–99%	<90%	0% (no OpenAPI spec, no versioning, no contract tests, no API linting)	High Risk

No additional hotspots beyond the standard set were observed.

4.4 Actions Required

Hotspot	Action	Rating	Priority
H1 — Dynamic Variable Creation	Replace 3 `extract()` calls with explicit variable assignment / typed DTOs; add CI rule to block future `extract()` usage	Moderate	High
H2 — API Sprawl	Author shared OpenAPI 3.1 spec; reconcile divergent endpoints (`bulk-import` , `checks` response shape) between Laravel and Express backends	High Risk	Medium
H3 — Missing API Governance	Introduce OpenAPI spec, API versioning (`/api/v1/`), Spectral linting, contract tests, `throttle` middleware, and tighten CORS	High Risk	Medium

4.5 Expected Outcomes

- Replacing `extract()` with explicit FormRequest validation and typed DTOs eliminates untraceable dynamic variable flow and closes the variable-shadowing injection vector in the public `carrierSummary` endpoint.
- A single OpenAPI 3.1 specification shared between Laravel and Express backends ensures both stacks serve identical contracts, enabling auto-generated TypeScript types for the frontend.
- API versioning (`/api/v1/`) provides a safe migration path for breaking changes without disrupting existing consumers.
- Contract tests (Spectral + Prism) running in CI catch endpoint divergence between the dual backends before code merges, preventing the silent drift already observed with `bulk-import` and `checks` .
- Rate limiting (`throttle` middleware) and tightened CORS (`allowed_origins` restricted to known frontend domains) reduce abuse surface on the currently wide-open API.","stop_reason":"end_turn","session_id":"12d12adf-073b-49db-9276-b352e8d7ba5d","total_cost_usd":1.9716114999999999,"usage":{"input_tokens":92,"cache_creation_input_tokens":97783,"cache_read_input_tokens":1088685,"output_tokens":17668,"server_tool_use":{"web_search_requests":0,"web_fetch_requests":0},"service_tier":"standard","cache_creation":{"ephemeral_1h_input_tokens":97783,"ephemeral_5m_input_tokens":0},"inference_geo":"not_available","iterations":[{"input_tokens":1,"output_tokens":1158,"cache_read_input_tokens":92449,"cache_creation_input_tokens":5334,"cache_creation":{"ephemeral_5m_input_tokens":0,"ephemeral_1h_input_tokens":5334},"type":"message"}],"speed":"standard"},"modelUsage":{"claude-haiku-4-5-20251001":{"inputTokens":7199,"outputTokens":16,"cacheReadInputTokens":0,"cacheCreationInputTokens":0,"webSearchRequests":0,"costUSD":0.007279,"contextWindow":200000,"maxOutputTokens":32000}
opus-4-6":{"inputTokens":92,"outputTokens":17668,"cacheReadInputTokens":1088685,"cacheCreationInputTokens":97783,"webSearchRequests":0,"costUSD":1.9643324999999998,"contextWindow":200000,"
[],"terminal_reason":"completed","fast_mode_state":"off","uuid":"e10e9ce7-98f7-4e43-b050-9e76849c66c3"}

---

5. Testing & Quality Assurance Analysis

EXECUTIVE SUMMARY

The Klearcom platform has a critically thin test suite. The backend (Laravel 12 / PHP 8.3) ships with PHPUnit 11 configured but contains only 2 test files with 3 test methods — none of which exercise any real application code; they assert hardcoded arrays and random integers. All 6 API controllers, both service classes (`RealTimeTestService` , `MongoService`), the `LegacyDataMapper` , and all 4 Eloquent models are completely untested. The frontend (React 19 / TypeScript / Vite) has zero test infrastructure — no Vitest, Jest, Testing Library, Cypress, or Playwright is installed, and zero test files exist across 14 source files. The Node.js dev-api (Express + MongoDB) likewise has zero tests and no test framework. CI runs `vendor/bin/phpunit` on the backend and `npm run build` on the frontend, but the frontend job only type-checks and builds with no test step at all. Estimated overall coverage is below 5%.

5.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	Good	Moderate	High Risk	Measured	Rating
H1	Untested Critical Logic	Critical modules with zero tests	0	1–3	>3	7	High Risk
H2	Low Test Coverage	Overall coverage %	>80%	50–80%	<50%	<5% (estimated, test-file-to-source ratio 2/35)	High Risk
H3	Missing Integration Tests	Boundaries covered %	>70%	30–70%	<30%	0% (no integration/feature tests exist)	High Risk
H4	Missing Contract Tests	APIs with contract tests %	>80%	40–80%	<40%	0% (0 of 15 endpoints)	High Risk
H5	Flaky / Skipped Tests	Skipped/flaky test count	0	1–5	>5	1 (non-deterministic `random_int` test)	Moderate
H6	No CI Test Gate	Tests enforced in CI	Required gate	Runs, not required	No CI test run	Backend: runs, not required; Frontend: no test run	Moderate
H7	No Frontend Test Infrastructure (additional)	Frontend test framework installed and tests present	Framework + tests	Framework, no tests	No framework	No framework installed, 0 test files	High Risk
H8	No Dev-API Tests (additional)	Dev-API test-file-to-source ratio	>50%	10–50%	<10%	0% (0 of 6 source files have tests)	High Risk
H9	Assertion-Free / Trivial Tests (additional)	Tests with no meaningful assertions	0	1–2	>2	2 (both existing test files assert only hardcoded values)	Moderate

5.4 Actions Required

Hotspot	Action	Rating	Priority
H1 — Untested Critical Logic	Add PHPUnit unit tests for `RealTimeTestService`, `MongoService`, `LegacyDataMapper`, and all 6 API controllers — prioritize state transition and reachability calculation logic	High Risk	Critical
H2 — Low Test Coverage	Enable PHPUnit coverage collection in CI; install Vitest in frontend and dev-api; target 50% in sprint 1, 75–80% before any major refactor	High Risk	Critical
H3 — Missing Integration Tests	Create `tests/Feature/` directory; add Laravel HTTP tests using `RefreshDatabase` against CI MariaDB; add MongoDB integration tests	High Risk	Critical
H4 — Missing Contract Tests	Add JSON Schema or snapshot contract tests for all 15 API endpoints; validate SSE event payloads match frontend type definitions	High Risk	Critical
H5 — Flaky / Skipped Tests	Replace `random_int()` test with deterministic test exercising real reachability calculation	Moderate	Medium
H6 — No CI Test Gate	Add `npm test` to frontend CI; add dev-api CI job; configure branch protection requiring all checks to pass	Moderate	High
H7 — No Frontend Test Infrastructure	Install Vitest + Testing Library + jest-dom; add `test` script; write tests for `useRealtimeTest`, `api/client.ts`, and page components	High Risk	Critical
H8 — No Dev-API Tests	Add Vitest + supertest to dev-api; test reachability logic and API contracts to prevent backend/dev-api divergence	High Risk	High
H9 — Assertion-Free Tests	Replace both trivial test files with tests that import and exercise real application classes	Moderate	Medium

5.5 Expected Outcomes

- **Critical paths protected:** Unit tests for `RealTimeTestService` and `MongoService` prevent regressions in IVR discovery, TFN reachability, and real-time streaming before they reach production.
- **Refactoring enabled safely:** Reaching 75–80% coverage provides the safety net needed to extract duplicated `buildTree()` logic, eliminate `extract()` usage, and modernize the `LegacyReportController`.
- **CI catches regressions automatically:** Adding test execution to all three CI jobs (backend, frontend, dev-api) with required status checks means broken code cannot merge.
- **Contract stability guaranteed:** Contract tests for the 15 API endpoints ensure that backend changes cannot silently break the frontend and that the dev-api stays in sync with production.
- **Developer confidence restored:** Replacing trivial/flaky tests with meaningful assertions provides an honest signal — green means the application works, red means it doesn't.","stop_reason":"end_turn","session_id":"91232d27-f403-4b5b-8254-08776bfe7e18","total_cost_usd":2.2005325,"usage":{"input_tokens":22,"cache_creation_input_tokens":98355,"cache_read_input_tokens":1159263,"output_tokens":25176,"server_tool_use":{"web_search_requests":0,"web_fetch_requests":0},"service_tier":"standard","cache_creation":{"ephemeral_1h_input_tokens":98355,"ephemeral_5m_input_tokens":0},"inference_geo":"not_available","iterations":[{"input_tokens":1,"output_tokens":1950,"cache_read_input_tokens":97909,"cache_creation_input_tokens":446,"cache_creation":

{"ephemeral_5m_input_tokens":0,"ephemeral_1h_input_tokens":446},"type":"message"}],"speed":"standard"},"modelUsage":{"claude-haiku-4-5-20251001":{"inputTokens":7756,"outputTokens":17,"cacheReadInputTokens":0,"cacheCreationInputTokens":0,"webSearchRequests":0,"costUSD":0.007841,"contextWindow":200000,"maxOutputTokens":32000},"claude-opus-4-6":{"inputTokens":22,"outputTokens":25176,"cacheReadInputTokens":1159263,"cacheCreationInputTokens":98355,"webSearchRequests":0,"costUSD":2.1926915,"contextWindow":200000,"maxOutput[],"terminal_reason":"completed","fast_mode_state":"off","uuid":"6b31ae44-04d1-4edd-853d-eb565845f641"}

---

6. Security Analysis

EXECUTIVE SUMMARY

The Klearcom platform presents a **High Risk** security posture driven by two critical vulnerabilities and systemic architectural gaps. The most severe finding is the complete absence of authentication or authorization on all 20+ API endpoints — every data-mutating and data-reading route is publicly accessible. A second critical finding is the use of PHP `extract()` on raw HTTP input in the legacy reporting path, which enables variable-injection attacks. Supporting high-severity issues include a fully permissive wildcard CORS policy (backend and dev-API), hardcoded database credentials committed to version control, an unvalidated bulk-import endpoint, and debug mode enabled in the Docker production config. On the positive side, dependency versions are current (Laravel 12, React 19, Vite 6), no known CVEs were found in declared dependencies, and the React frontend is free of XSS sinks (no `dangerouslySetInnerHTML`, `innerHTML`, or `eval` usage). Layers covered: backend (PHP/Laravel), frontend (React/TypeScript SPA), dev-API (Express/Node.js), infrastructure (Docker/nginx/CI).

6.1 Security Benchmark Ratings

#	Security KPI	Target	Good	Moderate	High Risk	Measured	Rating
H1	Critical Vulnerabilities	0	0	1	>1	2	High Risk
H2	High Vulnerabilities	0	<5	5–10	>10	4	Good
H3	Medium Vulnerabilities	low	<20	20–50	>50	6	Good
H4	Vulnerability Density	<0.5/KLOC	<0.5	0.5–1.0	>1.0	4.0/KLOC	High Risk
H5	OWASP Top 10 Compliance	>95%	>95%	80–95%	<80%	30%	High Risk
H6	Critical/High Vulnerable Deps	0	0	1	>1	0	Good
H7	Outdated Dependencies	<10%	<10%	10–25%	>25%	~0%	Good
H8	End-of-Life Dependencies	0	0	1–5	>5	0	Good

6.5 Actions Required

Finding	Action	Rating	Priority
No Authentication on API Routes	Add auth middleware (Sanctum/JWT) to all routes; implement RBAC and ownership checks	High Risk	Critical
PHP `extract()` on User Input	Replace `extract()` with explicit variable assignment; add input validation in `LegacyReportController`	High Risk	Critical
Wildcard CORS Configuration	Restrict `allowed_origins` to explicit frontend domain(s) in both backend and dev-API	High Risk	High
Hardcoded Database Credentials	Move credentials to `.env` / CI secrets; rotate exposed passwords	High Risk	High
Unvalidated Bulk Import Endpoint	Add input validation, array size limits, rate limiting, and authentication	High Risk	High
Debug Mode Enabled	Set `APP_DEBUG=false` in production/Docker configuration	High Risk	High
No Rate Limiting	Add `throttle:api` middleware (Laravel) and `express-rate-limit` (dev-API)	Moderate	Medium
No Security Headers	Configure CSP, HSTS, X-Frame-Options, X-Content-Type-Options in nginx	Moderate	Medium
Exposed Database Ports	Remove host port mappings or bind to 127.0.0.1; enable MongoDB auth	Moderate	Medium

No SAST/Dependency Scanning in CI	Add `composer audit`, `npm audit`, PHPStan, and secret scanning to CI pipeline	Moderate	Medium
Security Logging & Monitoring	Implement audit logging for API access and data changes; configure exception alerting	Moderate	Medium
Frontend HTTP Fallback	Default API URL to HTTPS or require explicit configuration	Moderate	Medium

---

7. Performance & Sustainability Analysis

EXECUTIVE SUMMARY

The Klearcom platform exhibits **High Risk** performance posture driven by compounding inefficiencies across the stack. The most severe issues are algorithmic: duplicated O(n²) recursive tree traversal in both the PHP backend and Node.js dev API re-scans the full node collection on every recursive call, degrading rapidly as IVR trees grow. API endpoints return unpaginated datasets, the dashboard fires 7–8 separate uncached SQL queries per page load, and SSE streaming re-fetches all historical events from MongoDB every 500 ms instead of using a cursor. Concurrency is critically impaired — all background work runs synchronously in PHP-FPM workers via `usleep` loops with no queue driver configured, limiting concurrent test throughput to the FPM pool size. On the infrastructure side, no Docker containers have resource limits, the CI pipeline recompiles the MongoDB PHP extension from source on every run with no dependency caching, Nginx serves all responses uncompressed, and the frontend ships a Vite dev server as its only runtime mode. Database volumes have no TTL or retention policy, ensuring unbounded storage growth.

7.1 Benchmark Ratings Summary

#	Hotspot	Primary KPI	Good	Moderate	High Risk	Measured	Rating
P1	Algorithm Efficiency	High-complexity algorithm sites	0	1–5	>5	6 (3 PHP + 3 Node.js)	High Risk
P2	Database Performance	Deferred → Backend Modernization (H14/H10)	—	—	—	See Backend Modernization	— (deferred)
P3	API Performance	Response-latency hotspots	0	1–5	>5	8	High Risk
P4	Memory Efficiency	High-memory sites	0	1–3	>3	4	High Risk
P5	CPU Efficiency	CPU-intensive operations on hot paths	0	1–5	>5	3	Moderate
P6	Concurrency	Parallelizable work + pool sizing (blocking-I/O → Backend Modernization H14)	0	1–5	>5	2 (sync dispatch + usleep workers)	Moderate
P7	Caching	Deferred → Backend Modernization H14 / Frontend Modernization H11	—	—	—	See those reports	— (deferred)
P8	Resource Utilization	Over-provisioned / idle resource configs	0	1–3	>3	5	High Risk
P9	Network Efficiency	Excessive-traffic sites	0	1–5	>5	5	Moderate
P10	Build Efficiency	Build/test pipeline efficiency	efficient	partial	slow / no caching	slow / no caching	High Risk
P11	Logging Efficiency	Excessive-logging sites	0	1–10	>10	3 (startup-only, low impact)	Good
P12	Sustainability	Resource-optimization posture	optimized	partial	wasteful	wasteful	High Risk

7.5 Actions Required

Hotspot	Action	Rating	Priority
P1 Algorithm Efficiency	Replace O(n²) recursive tree builds with single-pass parent-lookup map in both PHP and Node.js; consolidate duplicated implementations	High Risk	Critical
P3 API Performance	Consolidate dashboard KPIs into 2 cached queries; add pagination to all list endpoints; replace SSE full-refetch with cursor-based incremental query; remove duplicate checks query	High Risk	Critical
P8 Resource Utilization	Add resource limits and health checks to all containers; create production frontend Dockerfile; bind DB ports to localhost; add Compose profiles	High Risk	Critical

P10 Build Efficiency	Add CI dependency caching; use setup-php for MongoDB extension; remove npm install fallback; add multi-stage Docker builds and .dockerignore files; bake Composer install into image	High Risk	Critical
P4 Memory Efficiency	Add cursor offset to streaming event fetch; add column projection to tree queries; cap in-memory arrays in dev API and frontend	High Risk	High
P6 Concurrency	Configure Redis queue driver (Horizon); replace usleep loops with queued jobs with delays	Moderate	High
P12 Sustainability	Add MongoDB TTL indexes; implement MariaDB retention policy; create production frontend runtime; add Compose profiles; document data retention	High Risk	High
P5 CPU Efficiency	Track cursor offset to serialize only new events; replace raw setInterval with React Query refetchInterval; memoize formatted timestamps	Moderate	Medium
P9 Network Efficiency	Enable Nginx gzip; reduce polling during SSE streams; simplify MongoDB health check; set CORS max_age to 86400; add upstream keepalive	Moderate	Medium

7.6 Expected Outcomes

- **Lower-complexity algorithms** cut tree-display latency from O(n²) to O(n), enabling IVR trees with hundreds of nodes without degradation.
- **Cached and paginated API endpoints** reduce dashboard load from 8 SQL queries to 2 (cached), and bound payload sizes to predictable page sizes, cutting median API response times by 50–70%.
- **Queue-driven test execution** frees PHP-FPM workers from usleep blocking, increasing concurrent test throughput by an order of magnitude and enabling independent scaling of HTTP handling and test processing.
- **Cursor-based SSE streaming** eliminates redundant MongoDB re-fetches, reducing event-path database load by ~95% per active session and capping memory growth to O(new events) per poll.
- **Production Docker builds with resource limits, CI caching, and gzip** reduce container image sizes by 60–80%, CI run times by 30–60 seconds per push, and API payload sizes by 60–80%, while preventing resource starvation via container memory/CPU caps."},"stop_reason":"end_turn","session_id":"aa1b1e04-ae46-4370-b5db-20aea0014644","total_cost_usd":2.9377666000000002,"usage":{"input_tokens":4324,"cache_creation_input_tokens":97228,"cache_read_input_tokens":729536,"output_tokens":30311,"server_tool_use":{"web_search_requests":0,"web_fetch_requests":0},"service_tier":"standard","cache_creation":{"ephemeral_1h_input_tokens":97228,"ephemeral_5m_input_tokens":0},"inference_geo":"not_available","iterations":[{"input_tokens":1,"output_tokens":2104,"cache_read_input_tokens":97045,"cache_creation_input_tokens":183,"cache_creation":{"ephemeral_5m_input_tokens":0,"ephemeral_1h_input_tokens":183},"type":"message"}],"speed":"standard"},"modelUsage":{"claude-haiku-4-5-20251001":{"inputTokens":9131,"outputTokens":16,"cacheReadInputTokens":0,"cacheCreationInputTokens":0,"webSearchRequests":0,"costUSD":0.009211,"contextWindow":200000,"maxOutputTokens":32000}opus-4-6":{"inputTokens":4324,"outputTokens":30311,"cacheReadInputTokens":729536,"cacheCreationInputTokens":97228,"webSearchRequests":0,"costUSD":2.1164430000000003,"contextWindow":200000sonnet-4-6":{"inputTokens":29,"outputTokens":18851,"cacheReadInputTokens":586777,"cacheCreationInputTokens":94194,"webSearchRequests":0,"costUSD":0.8121126000000001,"contextWindow":200000,"m[],"terminal_reason":"completed","fast_mode_state":"off","uuid":"30a5e708-a4fe-42b8-a15f-193704dd6137"}